

# SAÉ 2.01 — Rapport individuel

| Neven | Quefelec | 2025–2026 | 12A

---

## 1. Présentation du projet

---

La SAÉ 2.01 consistait à développer une application Java en ligne de commande, `BriquesCollectionManager`, permettant à un collectionneur de LEGO® de gérer et d'explorer une base de données de boîtes, pièces, figurines et thèmes. L'architecture devait respecter une séparation claire en trois couches : métier (App), accès aux données (BD) et interface utilisateur (UI/console) soit l'architecture MVC.

## 2. Mes contributions

---

### Couche métier (package App) ou MODÈLE

J'ai développé les classes suivantes :

`Piece.java` et `PieceQuantite.java` : modélisation d'une pièce LEGO avec numéro, nom, catégorie et couleur, ainsi que l'association pièce/quantité/supplément utilisée dans le contenu des boîtes.

`Figurine.java` et `FigurineQuantite.java` : représentation des figurines avec leur identifiant, nom et nombre de parties, et leur quantité dans une boîte.

`Theme.java` et `ThemeService.java` : gestion des thèmes et sous-thèmes, incluant la méthode `rechercherBoitesParTheme()` qui parcourt récursivement la hiérarchie des thèmes pour retrouver toutes les boîtes associées.

### Couche accès aux données (package BD)

`PieceBD.java` : requêtes JDBC pour lister, rechercher, insérer, mettre à jour et supprimer des pièces ; jointure avec la table `CATEGORIE` pour enrichir les résultats.

`ThemeBD.java` : les méthodes d'accès aux données complète sur la table `THEME` avec gestion des relations parent/enfant (champ `idtheme_pere` nullable).

`ThemeParentBD.java` : méthodes spécialisées pour définir/récupérer le parent d'un thème et lister les sous-thèmes immédiats.

`FigurineBD.java` : gestion des figurines en base.

## Couche interface (package UI/console)

ConsoleUi.java : utilitaires d'affichage et de saisie (lireChoix, lireTexte, afficherTitre...).

AuthenticationService.java : flux de connexion et de création de compte, avec vérification de l'identifiant et confirmation du mot de passe.

PieceService.java : façade métier exposant la liste des pièces, catégories et couleurs à l'interface.

## 3. Difficultés rencontrées

---

### Requêtes JDBC et correspondance objet-relationnel

La plus grande difficulté du projet a été la mise en place des requêtes JDBC, notamment dans PieceBD et ThemeBD. La principale source d'erreurs venait des colonnes nullables : le champ idtheme\_pere peut être NULL pour un thème racine, et un mauvais appel à rs.getInt() au lieu de rs.getObject() provoquait silencieusement un 0 à la place de null, corrompant la hiérarchie des thèmes. Il a fallu systématiser le pattern (Integer) rs.getObject("idtheme\_pere") et ajouter un test explicite avant de construire l'objet parent.

La jointure LEFT JOIN entre PIECE et CATEGORIE a également posé problème : si une pièce n'avait pas de catégorie, la tentative de lire rs.getInt("idcat") levait une exception. J'ai résolu cela en vérifiant rs.getObject("idcat") != null avant d'instancier l'objet Categorie.

### Gestion des PreparedStatement et des ressources

Un autre point délicat a été la fermeture correcte des ressources JDBC. L'utilisation du try-with-resources sur PreparedStatement et ResultSet imbriqués n'était pas immédiatement intuitive, et plusieurs fuites de connexion ont été identifiées lors des tests avant d'adopter le pattern correct systématiquement.

## 4. Bilan et acquis

---

Cette SAÉ m'a permis de consolider ma maîtrise de JDBC, en particulier la gestion des valeurs nullables et la construction d'objets métier depuis des ResultSet. La séparation stricte des couches m'a forcé à réfléchir à la responsabilité de chaque classe et à éviter les dépendances croisées. La rédaction des tests JUnit sur les services m'a également appris à anticiper les cas limites (pièce inexistante, thème sans sous-thème, etc.).